JSON → TypeScript Type Generator

HackerRank

Given a JSON array of objects on **stdin**, output a deterministic TypeScript type declaration to **stdout**.

The output must match the expected answer **character-for-character** (including whitespace, ordering, and newlines).

- Use **LF** (`\n`) line endings (Unix-style). No `\r\n`.

Input Format

- **Line 1:** An integer `T` — the number of test cases ($1 \leq T \leq 50$).
- **Next 2×T lines:** For each test case, two lines:
  - **Line A:** A single string — the root type name (e.g., `RootType`). Contains no spaces or special characters.
  - **Line B:** A valid JSON array of objects, always on a **single line** (compact format). The array contains 0 to 10,000 objects.

Reading the input: Read the first line as the number of test cases. Then for each test case, read one line as the root type name and the next line as the compact JSON string. Parse the JSON using a standard JSON parser.

Constraints

- $0 \leq$ number of objects $\leq 10{,}000$
- JSON is valid and the top-level value is always an array.
- Every element of the top-level array is a JSON object `{...}` (never a primitive or array at the top level).
- Maximum nesting depth: 10 levels.
- Maximum total keys across all objects: 100,000.
- Key names: start with a lowercase letter `[a-z]`, followed by zero or more characters from `[a-zA-Z0-9_]`, and the **last character** is never a digit. In regex: single-char keys match `[a-z]`; multi-char keys match `[a-z][a-zA-Z0-9_]*[a-zA-Z_]`. (This ensures no key naturally generates a suffixed collision name like `Address2`.)
- The root type name (line 1 of input) matches `[A-Z][a-zA-Z]*` (starts with uppercase letter, letters only, no digits or special characters).
- No circular references (impossible in JSON).
- Array elements are never arrays themselves (no nested arrays like `[[1,2],[3,4]]`).
- If a field's value is an array containing objects in any record, that field's value is never a direct (non-array) object in any other record. (This avoids ambiguity in interface naming between element-level and field-level objects.)

Output Format

For each test case, print the TypeScript `.d.ts` declaration following the **exact formatting rules** below.

Separate the output of consecutive test cases with a single line containing exactly `---` (three hyphens, no spaces).

- No `---` before the first test case.
- No `---` after the last test case.
- **One trailing newline** after the very last closing brace of the last test case (i.e., the entire output ends with `}\n` or `{}\n`).

The judge performs a strict string comparison — there is only one correct output for any given input.

Formatting Rules (MUST follow exactly)

These rules guarantee a single deterministic output:

F1. Interfaces

- Each type is emitted as `export interface <Name> {` (one space between name and opening brace).
- Closing brace `}` is on its own line with no indentation.
- Non-empty interfaces: opening `{` on the declaration line, properties on subsequent lines, `}` alone on last line.
- Empty interfaces (no properties): `export interface RootType {}` (space before `{`, no space between `{` and `}`, all on one line).

F2. Properties

- Each property is on its own line, indented with **2 spaces**.
- Format: `<key><optional>: <type>;`
  - `<optional>` is `?` if the field is optional, otherwise empty.
  - A single space after the colon.
  - Terminated with a semicolon `;`.
- **Properties are sorted by case-sensitive ASCII order of the key name within each interface.**

F3. Type Expressions

- Primitive types: `string`, `number`, `boolean`, `null`.
- Union types: components sorted by **case-sensitive ASCII comparison** of their string representation, joined with `|` (space-pipe-space).
  - Sort key is the literal type string as written (e.g., `null`, `number`, `string`, `Address`, `number[]`, `(number | string)[]`).
  - Uppercase letters (A-Z, 65-90) sort before lowercase (a-z, 97-122).

    - Parenthesis `(` (40) sorts before all letters.
    - Example: `number | string` (NOT `string | number` because `n` < `s`).
    - Example: `boolean | null | number | string`.
    - Example with interface: `Address | null` (because `A` (65) < `n` (110)).
    - Example with parens: `(number | string)[] | boolean` (because `(` (40) < `b` (98)).
- Array types:
    - Single element type: `<type>[]` (e.g., `string[]`, `number[]`).
    - Union element type: `(<union>)[]` with parentheses (e.g., `(number | string)[]`).
    - Empty arrays with no other type info: `unknown[]`.
- Object types: Use the generated interface name (e.g., `Address`).
- Nullable object: `<InterfaceName> | null` (sorted by ASCII, so `Address | null` since `A` (65) < `n` (110)).

F4. Interface Ordering

- All interfaces are printed in **case-sensitive ASCII order by name** (uppercase A-Z before lowercase a-z; digits 0-9 before letters).
- Separated by exactly **one blank line** between each interface.
- No leading blank line before the first interface.
- **One trailing newline** after the last closing brace (i.e., the file ends with `}\n`).

F5. Naming (Deterministic — NO creative freedom)

- The root interface uses the **exact** name given on the first input line.
- **Every other interface is named after the KEY (property name) in the parent object whose value is that object (or array of objects).** The transform is: capitalize the first character of the key; leave all other characters unchanged.
    - Formula: `interfaceName = key[0].toUpperCase() + key.slice(1)`
    - The name is derived ONLY from the immediate parent key — NOT from ancestor path, NOT from the content/shape of the object.
- Examples:
    - Key `address` holds an object → interface name is `Address`
    - Key `userProfile` holds an object → interface name is `UserProfile`
    - Key `my_field` holds an object → interface name is `My_field`
    - Key `posts` holds an array of objects → interface for the element type is `Posts`
    - Key `x` holds an object → interface name is `X`
    - Nested example: `{ "user": { "profile": { "name": "Alice" } } }` → key `user` produces `User`, key `profile` produces `Profile` (NOT `UserProfile`).

- This is NOT full PascalCase or camelCase conversion. It is ONLY: `key[0].toUpperCase() + key.slice(1)`. No other transformation.

- **Name collisions:** If two different nested paths produce the same interface name after this transformation, OR if a derived name equals the root type name, append a numeric suffix starting at `2` for the collision: `Address`, `Address2`, `Address3`, etc.
  - The **root type name is reserved first** (it always gets the unsuffixed name).
  - Among derived interfaces, collision order is determined by **first encounter in a depth-first, alphabetical-key traversal** of the merged type tree. The first encountered gets the unsuffixed name (or `2` if root took it), subsequent get incrementing suffixes.
  - "Depth-first, alphabetical-key" means: at each object level, visit keys in sorted (ASCII) order; for each key, recurse into its children before moving to the next sibling key.
  - Name assignment uses a **global set of used names**. For each interface, try the base name; if taken, try base+`2`, base+`3`, etc. until an unused name is found. (Since keys never end in a digit, suffixed names like `Address2` can never naturally arise from a different key, so this is guaranteed collision-free.)

F6. Type Inference Rules

JSON value(s) observed	TypeScript Type
Any string	`string`
Any number (int or float)	`number`
`true` or `false`	`boolean`
`null`	`null`
Object `{...}`	Named interface
Array `[...]`	See array rules below

Each individual value maps to its type independently. When multiple types are observed for the same key, they form a union (see F7).

F7. Merging Rules (across all objects in the input array)

1. **Union of keys:** The output contains every key observed in any object.
2. **Optional (`?`):** A key is optional if it is **absent** from at least one object in the array.
   - A key that is present but `null` is NOT absent — it is present with type `null`.
3. **Type string computation:** For a key with mixed value types, see **R1** in the "Rules for Mixed/Complex Types" section below. The summary:
   - All arrays for that key are merged into ONE type string (using rule 6).
   - All objects for that key are merged into ONE interface.
   - Each distinct primitive/null type contributes its type string.
4. **Type union:** The field type is the sorted (ASCII) union of all distinct type strings (one for arrays if any, one for objects if any, plus each primitive type observed).
5. **Nested object merge:** If a key's value is an object in multiple objects, ALL those objects are merged into a single interface (recursive). The interface appears once in the type union.
6. **Array type string:** For arrays, collect ALL elements from ALL arrays for that key across ALL objects. Compute the union of type strings of elements:
   - If all elements produce one type string → `<type>[]` (e.g., `number[]`)

- If multiple distinct type strings → `(<sorted union>)[]` (e.g., `(number | string)[]`)
- If no elements seen (only empty arrays) → `unknown[]`
- If elements include objects, merge them into one interface (named from parent key); use the interface name as one of the union components.

7. **Array of objects:** If array elements include objects, ALL object elements across all arrays for that key are merged into one interface.

F8. Edge Cases

- **Empty input array `[]`:** Output a single empty interface with the given root name.
- **Object present in some, absent in others:** Field is optional, type is the interface name.
- **Object in some, null in some, absent in some:** Field is optional, type is `<Interface> | null`.
- **A field that is only ever `null`:** Type is `null`.
- **A field that is sometimes missing and sometimes `null`:** Optional with type `null` → `fieldName?: null;`
- **Field is array in some objects, non-array in others:** Not optional (if present in all); type is the union of all type strings (e.g., `number[] | string`).

Path Separation Rule

- Each unique path in the type tree produces its own interface, even if two paths produce interfaces with identical shapes.
- Example: `billing.address` and `shipping.address` are two different interfaces even if they have the same fields.
- Names are assigned per-path, with collision suffixes as needed.

Rules for Mixed/Complex Types

These cases are **valid input** and must be handled. The rules below guarantee a single correct output.

R1. Field has mixed categories across objects (e.g., array in one, primitive in another)

First, separate all observed values into categories: - **All arrays** for that key (across all objects) are merged together into ONE array type string (using F7.6: merge all elements into one pool). - **All objects** for that key are merged into ONE interface. - **Primitives/null** each contribute their type string.

Then compute the field type as the union of: - The single merged array type string (if any arrays exist): e.g., `number[]`, `(number | string)[]`, `unknown[]` - The single interface name (if any objects exist): e.g., `Config` - Each distinct primitive type observed: `string`, `number`, `boolean`, `null`

The union is sorted by ASCII.

Example: field `x` is `[1,2]` in obj 1, `["a"]` in obj 2, `true` in obj 3 → - Arrays merged: elements are `1, 2, "a"` → type string `(number | string)[]` - Primitive: `boolean` - Union sorted by ASCII: `(` (40) < `b` (98) → `(number | string)[] | boolean`

R2. Array elements mix objects with primitives

Collect ALL elements from ALL arrays for that field across all objects. For each element: - primitives/null → their type (`string`, `number`, `boolean`, `null`) - objects → merge ALL object elements into a **single interface** (named from parent key)

The element type is the union of the interface name + all primitive types observed, sorted by ASCII.

Example: field `items` has value `[1, {"a": true}, "hi"]` →
- Objects merged → interface `Items` with `{ a: boolean; }` - Primitives → `number`, `string` - Element type: `(Items | number | string)[]`

R3. Field is object in some, primitive/array in others

The objects at that path are merged into one interface (as usual). The field type is the union of all type strings.

Example: field `data` is `{"x":1}` in one object, `"raw"` in another, absent in a third →
- Interface `Data` with `{ x: number; }` - Type: `Data | string`, field is optional (absent in third object)
- Output: `data?: Data | string;`

R4. Field is object in some, null in some, and a primitive in others

Same union rule. The type is the sorted union of all observed type strings.

Example: field `meta` is `{"k":"v"}` in one, `null` in another, `42` in a third →
- Interface `Meta` with `{ k: string; }` - Type strings: `Meta`, `null`, `number` - ASCII sort: `M`(77) < `n`(110); then `null` vs `number`: compare char-by-char → `n`=`n`, `u`=`u`, `l`(108) < `m`(109) → `null` < `number` - Result: `Meta | null | number` - Output: `meta: Meta | null | number;`

Sample Input 0

```
13
RootType
[{"name":"Alice","age":30,"active":true},{"name":"Bob","age":25,"active":false}]
RootType
[{"id":1,"name":"Alice","email":"alice@example.com"},{"id":2,"name":"Bob"},
{"id":3,"name":"Charlie","email":null}]
RootType
[{"id":1,"name":"Alice","address":{"street":"123 Main St","city":"Springfield","zip":"62701"}},
{"id":2,"name":"Bob","address":{"street":"456 Oak Ave","city":"Shelbyville"}}]
RootType
[{"id":1,"tags":["admin","user"],"scores":[95,87,92]},{"id":2,"tags":["guest"],"scores":[78,81]},
{"id":3,"tags":[],"scores":[88],"metadata":{"source":"import"}}]
RootType
[{"id":1,"value":"hello","items":[1,"two",true]},{"id":"abc","value":42,"items":[null,3]}]
RootType
[{"user":{"profile":{"name":"Alice","avatar":
{"url":"https://img.example.com/a.png","width":100,"height":100}},"settings":
{"theme":"dark","notifications":true}},"posts":[{"title":"Hello","likes":5},
{"title":"World","likes":12,"pinned":true}]},{"user":{"profile":{"name":"Bob","avatar":null},"settings":
{"theme":"light","notifications":false,"language":"en"}},"posts":[]}]
RootType
[]
RootType
[{"a":null,"b":1},{"a":null,"b":null},{"b":2}]
RootType
[{"billing":{"address":{"city":"NYC"}},"shipping":{"address":{"city":"LA","zip":"90001"}}}]
```

```
Data
[{"config":{"retries":3}},{"config":null},{}]
RootType
[{"data":[1,2,3]},{"data":"raw"}]
RootType
[{"items":[1,{"label":"x"},"hello",{"label":"y","count":5}]}]
Config
[{"config":{"timeout":30},"name":"app"}]
```

Sample Output 0

```
export interface RootType {
  active: boolean;
  age: number;
  name: string;
}
---
export interface RootType {
  email?: null | string;
  id: number;
  name: string;
}
---
export interface Address {
  city: string;
  street: string;
  zip?: string;
}

export interface RootType {
  address: Address;
  id: number;
  name: string;
}
---
export interface Metadata {
  source: string;
}

export interface RootType {
  id: number;
  metadata?: Metadata;
  scores: number[];
  tags: string[];
}
---
export interface RootType {
  id: number | string;
  items: (boolean | null | number | string)[];
  value: number | string;
}
---
export interface Avatar {
  height: number;
  url: string;
  width: number;
}

export interface Posts {
  likes: number;
  pinned?: boolean;
  title: string;
}

export interface Profile {
  avatar: Avatar | null;
  name: string;
}
```

```
export interface RootType {
  posts: Posts[];
  user: User;
}

export interface Settings {
  language?: string;
  notifications: boolean;
  theme: string;
}

export interface User {
  profile: Profile;
  settings: Settings;
}
---
export interface RootType {}
---
export interface RootType {
  a?: null;
  b: null | number;
}
---
export interface Address {
  city: string;
}

export interface Address2 {
  city: string;
  zip: string;
}

export interface Billing {
  address: Address;
}

export interface RootType {
  billing: Billing;
  shipping: Shipping;
}

export interface Shipping {
  address: Address2;
}
---
export interface Config {
  retries: number;
}

export interface Data {
  config?: Config | null;
}
---
export interface RootType {
  data: number[] | string;
}
---
export interface Items {
  count?: number;
  label: string;
}

export interface RootType {
  items: (Items | number | string)[];
}
---
export interface Config {
  config: Config2;
  name: string;
}
```

```
export interface Config2 {
  timeout: number;
}
```